

TSIGE - Bases de Datos para Ciudades 3D

Nicolás Almará
nicoalmar11@gmail.com

Facundo Bacigalupi
bacigalupifacundo@gmail.com

Viviana Escobar
viviana.escobar@fing.edu.uy

Christian Ferrari
christian.ferrari@fing.edu.uy

Michel Guerrero
michel.guerrero@fing.edu.uy

Agustín Koltukian
aguskoltukian@gmail.com

Resumen

Hoy en día, la gestión y planificación de las ciudades exige dejar atrás los mapas tradicionales en dos dimensiones para pasar a un análisis tridimensional real, sobre todo al momento de controlar normativas urbanas complejas como las alturas o los volúmenes construidos. Este trabajo presenta el diseño y desarrollo de un prototipo de Sistema de Información Geográfica (GIS) empresarial enfocado en evaluar de forma automática estas reglas territoriales sobre un modelo 3D real de la ciudad de Nueva York. Para lograrlo, cargamos datos en el estándar semántico CityGML 2.0 dentro de la plataforma 3D CityDB, la cual corre sobre un motor PostgreSQL con la extensión espacial PostGIS para resolver los cálculos volumétricos y las consultas espaciales de fondo. Además, cruzamos este modelo con el dataset catastral MapPLUTO para validar indicadores clave en las parcelas, como el Coeficiente de Edificabilidad (FAR) y las restricciones de altura por zona. Los resultados de las pruebas demuestran que es totalmente viable usar bases de datos geoespaciales 3D para fiscalizar el desarrollo urbano de manera automatizada, exponiendo todas estas métricas a través de un visor web interactivo basado en geoservicios estándar de la OGC.

Palabras Clave

GIS, Ciudades 3D, CityGML, 3D CityDB, PostGIS, GeoServer, Normativa Urbana, MapPLUTO

1 Introducción

En el contexto del desarrollo de las *Smart Cities*, contar con una representación digital precisa de los edificios y el entorno urbano se volvió fundamental para los gobiernos locales y los encargados de la planificación territorial. Por lo general, los registros de catastro y las normativas de ordenamiento urbano se han manejado a través de mapas tradicionales en dos dimensiones (2D). Sin embargo, los planes de zonificación actuales imponen restricciones complejas que involucran directamente la tercera dimensión, como los límites de altura según el contexto de la manzana, la ocupación basada en volúmenes (FAR), los conos de sombra o las servidumbres de luz en zonas con alta densidad de población.

El principal problema técnico es que los modelos 3D puramente visuales o de diseño gráfico no tienen la semántica necesaria para entender qué representa cada parte de la estructura (por ejemplo, no pueden diferenciar una pared de un techo o vincular un edificio a su parcela catastral). Para solucionar esto, el Open Geospatial Consortium (OGC) [9] diseñó el estándar CityGML [7], que permite guardar modelos de ciudades combinando la geometría con propiedades semánticas orientadas a objetos.

El objetivo del proyecto es evaluar qué tan viable es usar la plataforma de código abierto 3D CityDB versión 5.1.3 [1] junto con el motor de bases de datos PostgreSQL y su extensión espacial

PostGIS [11] para almacenar y analizar estos modelos urbanos tridimensionales.

Como caso de estudio, tomamos un fragmento del modelo de edificios en 3D con Nivel de Detalle 2 (LoD2) de la ciudad de Nueva York provisto por la Universidad Técnica de Múnich (TUM) [3]. Aunque los archivos originales están en formato CityGML 2.0, el motor de la base de datos los mapea de forma interna al nuevo esquema de CityGML 3 [1]. Toda esta información geográfica se unificó bajo el sistema de referencia SRID 32118 (NAD83 / New York Long Island) [5] y se cruzó mediante consultas espaciales nativas con los datos de zonificación catastral de MapPLUTO, desarrollados por el Departamento de Planificación Urbana de Nueva York (DCP) [6].

El sistema que desarrollamos no solo calcula las infracciones de altura o los coeficientes de edificabilidad a nivel interno mediante lenguaje SQL, sino que además expone estos datos procesados a través de geoservicios Web Feature Service (WFS)[8] en GeoServer usando el formato GeoJSON [10]. Por último, un cliente web empresarial consume estos servicios y utiliza la librería *CesiumJS* para renderizar en el navegador del usuario los polígonos extruidos por altura en un entorno 3D completamente interactivo [2].

2 Marco Conceptual

Para poder armar el prototipo y procesar las métricas propuestas, primero vale aclarar los conceptos y herramientas tecnológicas seleccionadas para manejar los datos en tres dimensiones.

2.1 Modelos de Ciudades 3D y el Estándar CityGML

Los modelos urbanos para GIS necesitan combinar la geometría con datos reales de lo que representa cada objeto. Para esto se usa CityGML [7], que es un estándar abierto basado en XML diseñado por la OGC para compartir modelos de ciudades virtuales.

Lo bueno de CityGML es que trabaja orientado a objetos. Esto significa que la base de datos reconoce clases específicas como edificios (*Building*), calles (*Road*) o parcelas de uso de suelo. Además, en los niveles detallados permite desglosar un edificio en partes con significado propio, separando los techos (*RoofSurface*), las paredes (*WallSurface*) y el suelo (*GroundSurface*).

El estándar también maneja el concepto de Niveles de Detalle (LoD, por sus siglas en inglés), que van del LoD0 al LoD4 según qué tan preciso sea el modelo. Para este proyecto se utilizan datos en **LoD2** [3]. Este nivel proporciona la estructura en 3D con la forma real de los techos. Aunque los archivos fuente originales vienen en la versión CityGML 2.0, la herramienta 3D CityDB los importa y los traduce internamente al esquema lógico de CityGML 3 [1].

2.2 Bases de Datos Geoespaciales 3D y PostGIS

Trabajar con archivos GML puros no es viable para un sistema empresarial porque son muy pesados, difíciles de consultar en simultáneo y no tienen índices rápidos. La solución es poner toda esa información en una base de datos relacional que soporte datos espaciales.

Se decidió utilizar PostgreSQL junto con su extensión **PostGIS** [11] que permite guardar las geometrías, crear índices espaciales (tipo R-Tree) y hacer consultas usando funciones SQL nativas. Adicionalmente, el entorno se configuró usando el sistema de referencia proyectado SRID 32118, asegurando que los cálculos de áreas y distancias coincidan con la cartografía oficial de Nueva York.

Para el procesamiento espacial, se utilizó la extensión nativa PostGIS, aprovechando sus funciones de análisis geométrico para vincular las parcelas bidimensionales con las alturas registradas.

2.3 3D City Database (3D CityDB)

3D CityDB [1] es un software libre que crea automáticamente un esquema de tablas interconectadas en nuestra base de datos (en el esquema citydb) preparado para la lógica de CityGML. Tablas como citydb.feature o citydb.geometry_data centralizan los componentes espaciales de la ciudad. La carga de datos se realiza mediante la herramienta *CityDB-tool*, ejecutada desde la consola de comandos en un contenedor Docker. Este procedimiento automatiza el procesamiento de la información y configura los índices espaciales requeridos para optimizar el rendimiento de las consultas.

2.4 Arquitectura Web GIS y Geoservicios Estándar OGC

Para visualizar de manera interactiva los resultados obtenidos a partir de las consultas SQL, tales como mapas de densidad y alertas de infracciones urbanísticas, se implementó una arquitectura web basada en geoservicios. Como componente central se utilizó **GeoServer** [10], desplegado en un contenedor Docker y conectado directamente a la base de datos PostgreSQL/PostGIS.

GeoServer publica las capas analíticas mediante estándares definidos por el OGC. En particular, se empleó el servicio **WFS (Web Feature Service)**, que permite exponer entidades geoespaciales junto con sus atributos alfanuméricos en formato GeoJSON. De esta manera, el cliente puede acceder a la información procesada, incluyendo métricas y atributos tales como identificadores de edificios, propietarios, alturas registradas y zonas de zonificación.

La interfaz web se despliega mediante un servidor Nginx encargado de servir los recursos estáticos de la aplicación. En el lado del cliente, la biblioteca de código abierto **CesiumJS** [2] consume los datos GeoJSON publicados por GeoServer y los representa sobre un entorno tridimensional. A partir de la información geométrica y de los atributos calculados, CesiumJS genera polígonos proyectados en tres dimensiones según su altura correspondiente empleando la GPU del cliente. Como resultado, se obtiene un visor web interactivo en 3D capaz de representar grandes volúmenes de información geoespacial con un alto nivel de fluidez y sin requerir la instalación de software adicional.

3 Descripción del Problema

Las normativas urbanas modernas introducen restricciones de naturaleza tridimensional. Parámetros como la altura máxima permitida en distritos residenciales, la ocupación del suelo o el Coeficiente de Edificabilidad (FAR, por sus siglas en inglés) basado en el volumen total construido, no se pueden calcular de forma precisa o automatizada si solo miramos el área de un edificio en un mapa catastral.

Para resolver esto, es necesario migrar hacia modelos urbanos en 3D que contengan semántica (es decir, que la base de datos sepa qué parte es un techo, qué es una pared y a qué lote pertenecen).

Al intentar implementar una solución para la ciudad de Nueva York, surgen tres problemas fundamentales:

- (1) **El volumen masivo de los datos fuente:** El dataset completo de edificios en 3D de Nueva York supera los 31 GB de almacenamiento y contiene más de un millón de estructuras. Procesar archivos XML/GML de este tamaño en computadoras comunes consume la memoria y ralentiza cualquier sistema de archivos tradicional, lo que hace inviable su uso sin una base de datos geoespacial especializada.
- (2) **La desconexión entre el mundo 3D y el catastral:** Los modelos 3D de los edificios (obtenidos de la Universidad Técnica de Múnich [3]) y los datos normativos/catastrales de los lotes (obtenidos del dataset MapPLUTO del departamento de planificación de la ciudad [6]) vienen de fuentes separadas y en formatos distintos. El problema surge de cómo cruzar de forma eficiente la geometría tridimensional del edificio con los atributos alfanuméricos de la normativa del lote (como las zonas residenciales tipo R6B), requiriendo un diseño de base de datos interconectado.
- (3) **El rendimiento en la visualización web:** Renderizar miles de edificios en 3D con geometrías complejas (techos inclinados, detalles de fachadas) directamente en un navegador web requiere mucha capacidad de cómputo (GPU) en el cliente y satura el ancho de banda al transferir los datos desde el servidor. El desafío está en encontrar un equilibrio técnico para poder mostrar un mapa interactivo fluido sin perder la precisión de los datos de altura y volumen necesarios para el análisis.

Para que el proyecto sea viable dentro de los recursos de infraestructura disponibles, se acotó el problema a una zona de estudio específica (dentro de los barrios de Middle Village y Glendale, en Queens/Brooklyn) definida por el encuadre espacial de la coordenada BBOX (EPSG:32118: 308000, 60500, 309500, 62000). El recorte sobre el dataset real completo de la ciudad resultó en una muestra de 1.597 edificios. Por otro lado, los datos cargados de MapPLUTO son de 856.614 lotes catastrales, lo que garantiza que la muestra del dataset tiene los datos completos.

4 Solución Planteada

Para resolver los desafíos asociados al volumen de datos, la integración de información catastral y el rendimiento de las aplicaciones web, se diseñó un Sistema de Información Geográfica (GIS, por sus siglas en inglés) empresarial e interconectado. La solución permitió automatizar los procesos de fiscalización territorial mediante

la integración de información estructural de los edificios con las normativas de ordenamiento urbano vigentes.

La solución desarrollada no consiste en procesar los archivos masivos de forma aislada, sino en delegar la inteligencia del sistema en una base de datos geoespacial robusta y estructurar una secuencia de servicios web eficientes.

La solución propuesta se estructura en tres componentes principales:

- (1) **Acotación y centralización en Base de Datos Espacial:** La arquitectura se fundamenta en la importación de los datos a un repositorio geoespacial centralizado basado en PostgreSQL/PostGIS mediante 3D CityDB. Para reducir los requerimientos computacionales asociados al procesamiento del modelo urbano completo, se definió una zona de estudio controlada de 2.25 km² ubicada entre los distritos de Queens y Brooklyn.
- (2) **Cruce normativo automatizado mediante Vistas SQL:** La vinculación entre el modelo urbano tridimensional de la TUM y la información normativa obtenida de MapPLUTO se implementó mediante vistas analíticas desarrolladas sobre PostGIS. Estas vistas ejecutan operaciones espaciales de intersección (ST_Intersects) entre los centroides de los edificios y las parcelas catastrales correspondientes, permitiendo determinar automáticamente las infracciones de altura y las estimaciones volumétricas (FAR) multiplicando el área proyectada de la huella (ST_Area) por la altura del bloque.
- (3) **Optimización del flujo web mediante doble modo de visualización:** Con el fin de equilibrar capacidad analítica y desempeño gráfico, se implementaron dos modalidades complementarias de visualización. La primera, denominada *LoD1 Analytics* utiliza las huellas base de los edificios almacenadas en la tabla `public.zona_analytics` y publicadas por GeoServer mediante servicios WFS en formato GeoJSON. Esta modalidad permite al usuario cambiar dinámicamente la simbología para colorear por zona normativa, incumplimientos de FAR o exposición solar. La segunda modalidad, *LoD2 Real 3D* emplea las geometrías tridimensionales completas de los edificios convertidas al formato *3D Tiles*, lo que posibilita una representación masiva y eficiente de la ciudad en el navegador. Los atributos asociados a cada entidad se recuperan bajo demanda mediante consultas WFS iniciadas por la interacción del usuario.

Este enfoque permite combinar visualizaciones ligeras orientadas al análisis espacial con representaciones tridimensionales detalladas, manteniendo simultáneamente un elevado rendimiento de renderizado y la precisión de la información analítica. La Figura 1 presenta el diagrama de arquitectura y la figura 2 el flujo de trabajo de la solución implementada.

4.1 Gestión y Tratamiento de Datos

El núcleo analítico de la solución propuesta se fundamenta en la convergencia y transformación de dos flujos de datos de naturaleza heterogénea. El trabajo realizado con los datos no se limita a un almacenamiento pasivo, sino que implica un pipeline conceptual de filtrado, enriquecimiento semántico y optimización geométrica:

- **Ingesta y Adecuación del Modelo Edificio (CityGML):** Se parte del dataset de estructuras 3D en nivel de detalle LoD2 [?]. El trabajo principal aquí consiste en una operación de *filtrado por streaming* para aislar el volumen masivo original (31 GB) a la zona de estudio (BBOX) sin saturar la memoria del host [? ? ?]. Además, se realiza un proceso crítico de *normalización de cotas (z)*, trasladando los vértices de los techos y paredes respecto al nivel mínimo del suelo de cada edificio para corregir el desfase con el mapa base y evitar que las estructuras floten en el visor [?].
- **Procesamiento y Normalización Catastral (MapPLUTO):** Los datos alfanuméricos y vectoriales de las parcelas de Nueva York se importan y se someten a un proceso de *reproyección cartográfica* hacia el sistema de coordenadas local EPSG:32118 [?]. Esto es vital para garantizar que las geometrías de los lotes y las huellas de los edificios hablen el mismo "idioma métrico" antes de realizar cualquier cálculo espacial [?].
- **Pipeline de Cruce Espacial e Indicadores Analíticos:** Una vez unificados los conjuntos en la base de datos, el trabajo de integración se ejecuta mediante SQL espacial nativo. En lugar de procesar mallas tridimensionales complejas, se aplica una optimización matemática: se calcula el centroide de la huella basal de cada edificio y se interseca geoméricamente (ST_Intersects) con los polígonos de las parcelas catastrales [?]. Al lograr esta vinculación semántica, el sistema calcula de forma automatizada:
 - (1) El volumen estimado de la edificación multiplicando el área proyectada de su plano base (ST_Area) por su altura registrada [?].
 - (2) Las infracciones normativas mediante la contrastación del volumen obtenido contra el Coeficiente de Edificabilidad (FAR) permitido por la zona catastral [? ?].

5 Arquitectura del Sistema

Para dar soporte a la solución planteada, se diseñó una arquitectura web GIS empresarial distribuida en tres capas funcionales independientes: almacenamiento espacial, servidores de geoservicios y la interfaz de visualización de usuario. Esta separación lógica garantiza la escalabilidad del sistema y desacopla el procesamiento pesado de datos de la lógica de renderizado en el cliente.

A nivel de infraestructura física, el entorno se implementó mediante la virtualización de componentes utilizando contenedores Docker [4], distribuyendo las responsabilidades en una red local aislada e interconectada:

- **Contenedor citydb (PostgreSQL 18.3 [12] / PostGIS 3.6.2):** Guarda la base de datos del laboratorio. Ejecuta el esquema estructurado de 3D CityDB v5 [1] bajo el sistema de referencia proyectado EPSG:32118.
- **Contenedor geoserver (GeoServer v2.25):** Actúa como el servidor de mapas oficial del sistema. Se conecta de forma directa al esquema público de PostgreSQL para leer las vistas analíticas generadas y exponerlas hacia la web mediante el estándar WFS de la OGC [10].

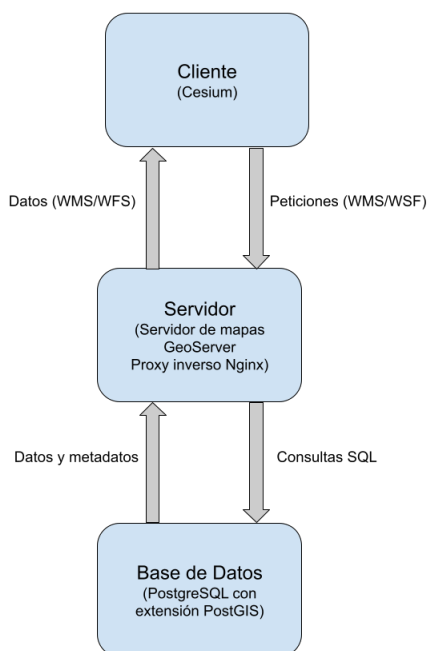


Figure 1: Diagrama de componentes y arquitectura lógica de alto nivel del sistema.

- **Contenedor cesium-viewer (Nginx):** Un servidor web liviano configurado para servir los archivos estáticos de la aplicación (HTML, CSS y JavaScript) y las estructuras de archivos de los *3D Tiles* almacenadas en el disco.

5.1 Flujo de Importación y Procesamiento de Datos

El procesamiento de la información se realiza localmente mediante scripts automatizados diseñados para optimizar el consumo de memoria del host, siguiendo los pasos definidos en el diseño del flujo de trabajo:

- (1) **Descarga y Filtrado por Streaming:** Mediante el script `filter_citygml_bbox.py`, el sistema lee por *streaming* el archivo comprimido original de los edificios (cuyo GML interno supera los 31 GB) utilizando la herramienta 7z. El script procesa las etiquetas XML y genera un archivo llamado `NYC_Buildings_LoD2_bbox.gml`, aislando únicamente los elementos dentro de la zona de análisis de Queens y Brooklyn.
- (2) **Carga en Base de Datos:** Este CityGML acotado se inyecta en el esquema de la base de datos utilizando la consola de comandos `citydb-tool`. En paralelo, los lotes de la herramienta catastral MapPLUTO [6] se importan, se reproyectan a EPSG:32118 y se recortan usando la herramienta de consola `ogr2ogr` (GDAL).
- (3) **Cálculo de Indicadores (Pipeline analítico):** Con toda la información unificada en las tablas, el script `vistas_sql.sql` procesa mediante funciones de PostGIS

las intersecciones espaciales y las reglas normativas, poblando la tabla principal del visor analítico (`public.zona_analytics`) con 1.597 registros listos para consumir.

5.2 Publicación e Interfaz de Visualización

Una vez procesados los datos, la arquitectura separa la salida en dos canales independientes para optimizar la carga web. El visor web realiza consultas dinámicas y asíncronas basadas en la navegación y eventos del usuario para optimizar el ancho de banda y garantizar la fluidez del renderizado:

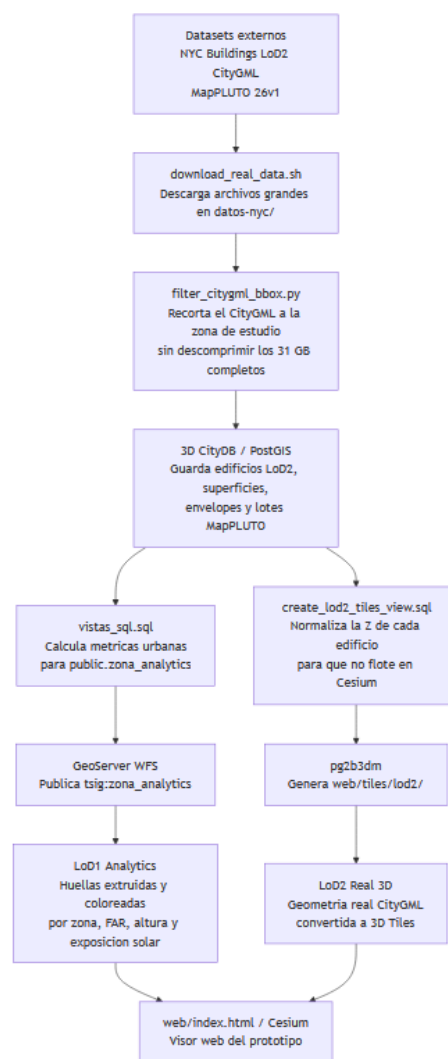


Figure 2: Diagrama global del flujo de datos del sistema.

- **Canal Analítico (WFS / GeoJSON):** GeoServer toma la tabla `zona_analytics` y la publica como un servicio WFS bajo la capa `tsig:zona_analytics`[cite: 133]. Cuando el

usuario navega por el mapa, el visor CesiumJS realiza peticiones HTTP asíncronas delimitadas por el encuadre visual de la pantalla (BBOX). El servidor responde únicamente con las huellas GeoJSON de los edificios visibles en ese cuadrante, encargándose el navegador de pintar y proyectar verticalmente las estructuras en 3D según los indicadores seleccionados por el usuario.

- **Canal Geométrico Real (3D Tiles bajo demanda):** Para el modo de visualización realista, primero utilizamos el script `create_lod2_tiles_view.sql` para preparar una vista en la base de datos que contiene la geometría 3D nativa de los techos y paredes de los edificios. Luego, la herramienta de consola `pg2b3dm` se conecta a esta vista, transforma las filas geográficas y genera archivos binarios jerárquicos (*3D Tiles*). Estos archivos resultantes se alojan en la carpeta estática del servidor Nginx (`web/tiles/lo2/`). CesiumJS evalúa constantemente el nivel de detalle jerárquico visible (HLOD) y solicita selectivamente al servidor los archivos `.b3dm` específicos de la zona observada, optimizando el uso de la GPU.
- **Peticiones Alfanuméricas por Interacción (GetFeature):** Con el fin de no sobrecargar el renderizado tridimensional con datos tabulares masivos, los atributos detallados de la normativa (como el FAR permitido por MapPLUTO) no se embeben en la geometría. Cuando el usuario hace clic sobre un edificio en la interfaz, CesiumJS captura el identificador único de la estructura y dispara una consulta puntual WFS de tipo `GetFeature` a GeoServer. El servidor procesa la petición y devuelve los atributos alfanuméricos en formato JSON, los cuales se despliegan inmediatamente en un panel emergente en el cliente.

6 Implementación

La implementación se realizó mediante la automatización de scripts en Bash y Python que coordinan los contenedores Docker, asegurando que cualquier miembro del equipo pueda replicar el entorno desde cero con el comando `docker compose up -d`.

6.1 Productos y Herramientas

Para el despliegue del sistema se evaluaron y utilizaron diversas herramientas de software libre y código abierto. A continuación, en la Tabla 1 se presenta un análisis del rol de cada una en el proyecto.

6.2 Problemas Encontrados

Durante las etapas de importación, normalización y despliegue del prototipo, se identificaron los siguientes desafíos técnicos:

- **Procesamiento de archivos masivos (Streaming con 7z):** El archivo XML/GML original de los edificios LoD2 de Nueva York superaba los 31 GB una vez descomprimido, excediendo la capacidad de memoria disponible para su procesamiento directo. Para evitar la necesidad de almacenar y manipular el archivo completo en disco se utilizó el script `filter_citygml_bbox.py`. Este script utiliza la herramienta 7z (o p7zip) para acceder al contenido comprimido y procesarlo mediante un flujo continuo de datos (*streaming*). Durante la lectura, se evalúa la ubicación espacial de

Table 1: Evaluación de Productos y Herramientas

Producto	Puntos Fuertes / Débiles	Evaluación
3D CityDB [1]	(+) Esquema optimizado para CityGML. (-) Curva de aprendizaje empinada.	Excelente. Evita diseñar las tablas 3D desde cero.
PostgreSQL+ PostGIS [11]	(+) Índices R-Tree rápidos. (-) Alto consumo de CPU en mallas 3D.	Indispensable. Motor central del pipeline analítico.
GeoServer [10]	(+) Publicación WFS y GeoJSON liviano. (-) No optimizado para mallas 3D nativas.	Muy bueno. Intermediario analítico eficiente.
CesiumJS [2]	(+) Renderizado nativo con WebGL y 3D Tiles. (-) Exige buena GPU en el cliente.	Excelente. La mejor librería web para globos 3D.

cada edificio y se verifica si sus coordenadas se encuentran dentro de la región de interés definida por la BBOX de estudio en Queens y Brooklyn (EPSG:32118: 308000, 60500, 309500, 62000). Si el edificio está dentro de la zona, se incorpora al archivo resultante `NYC_Buildings_LoD2_bbox.gml`, mientras que el resto se descarta durante el proceso de lectura.

- **Edificios flotantes en el mapa base:** Durante las pruebas de visualización tridimensional en CesiumJS se observó que muchos edificios aparecían suspendidos sobre la superficie del terreno. Esto se debía a que los datos proporcionados por la TUM [3] utilizan alturas absolutas referidas sobre el nivel del mar, mientras que el mapa base bidimensional de CesiumJS se representa sobre una superficie plana con elevación de referencia igual a cero. Para solucionarlo, se desarrolló el script `create_lod2_tiles_view.sql` que calcula el valor de elevación mínima (`z_min`) de cada estructura y normaliza sus coordenadas verticales mediante la traslación de todos sus vértices respecto de dicho valor, logrando que los edificios tengan su base sobre el piso.
- **Limitaciones en la representación de capas urbanas secundarias:** Al finalizar la importación de los datos, se detectó que la tabla analítica de calles (`public.zona_roads`)

no contenía registros. Al investigar el comportamiento de la herramienta de importación, se identificó que el estándar 3D CityDB unifica calles y plazas bajo una clase genérica perdiendo la distinción fina de los archivos de origen, sumado a que el dataset de la TUM no incluía zonas verdes. Como consecuencia, las métricas desarrolladas se acotaron estrictamente al análisis de edificaciones y parcelas catastrales.

7 Evaluación de la Solución

8 Desarrollo del Proyecto

El proyecto se desarrolló a lo largo de un período de 8 semanas. De acuerdo con la planificación sugerida del curso, el cronograma inicial preveía una distribución equitativa de dos semanas para cada una de las cuatro fases principales: Relevamiento y Diseño, Procesamiento y Preparación de Datos, Pipeline Analítico SQL, y Visualización e Informe.

Si bien el informe estaba planificado para empezarse recién en la última fase, el equipo adoptó una estrategia iterativa. Una vez establecido el dataset de Nueva York con el que queríamos trabajar, y luego de verificar que su uso era completamente viable y satisfactorio para los objetivos del laboratorio, se comenzó de forma anticipada con la redacción del documento. Esto permitió definir y estructurar con certeza, desde las primeras semanas, las herramientas del stack tecnológico que ya estaban confirmadas (como PostgreSQL, PostGIS y el dataset catastral MapPLUTO).

8.1 Análisis del Proceso y Desafíos Técnicos

En la práctica, la mayor carga de trabajo se decantó hacia la resolución de anomalías en los datos y la configuración de la interfaz:

- **Ingesta y Adecuación de Datos:** La etapa de preparación demandó un gran esfuerzo debido al volumen masivo del CityGML original y a la necesidad de diseñar el filtrado por streaming. A esto se sumó la resolución del problema de los edificios flotantes, lo que exigió desarrollar un pipeline analítico intermedio para normalizar las cotas verticales (z) antes de poder consumir la información.
- **Puesta en Marcha de CesiumJS y Visualización:** Lo que llevó más tiempo y dedicación en la construcción del prototipo fue configurar el entorno de CesiumJS y lograr un renderizado fluido de las estructuras en el navegador una vez recortada la zona de interés de Queens y Brooklyn.
- **Abstracción del Pipeline Analítico:** En paralelo con la visualización, una parte importante del trabajo se concentró en el "pienso" y diseño de las consultas espaciales nativas. Se invirtió tiempo clave en el diseño conceptual de las vistas para que los cálculos urbanísticos complejos quedaran completamente seteados y optimizados en la base de datos, garantizando una respuesta inmediata ante la interacción del usuario.
- **Despliegue de Servidores:** A diferencia de la complejidad geométrica de las fases previas, la puesta en marcha de la red interna de contenedores y la configuración de Nginx como servidor inverso se ejecutaron de manera fluida. Esto fue posible gracias a la división de roles dentro del equipo, delegando estas tareas en un perfil especializado en administración de configuración (SCM).

9 Conclusiones y Trabajo a Futuro

9.1 Conclusiones

A partir del diseño, desarrollo e implementación del prototipo para la ciudad de Nueva York, es posible concluir que la utilización de bases de datos geoespaciales tridimensionales bajo estándares internacionales es una solución tecnológica completamente viable, robusta y razonable para la fiscalización y control del desarrollo urbano. El proyecto cumplió exitosamente con el objetivo general planteado al inicio del laboratorio, el cual era automatizar el cruce de datos volumétricos detallados y semánticos provenientes del modelo edilicio con las reglas normativas y alfanuméricas del catastro territorial.

La experiencia práctica demostró que el uso integrado de herramientas de código abierto como PostgreSQL con PostGIS, la estructura de almacenamiento de 3D CityDB y la exposición mediante servicios estándar de la OGC (WFS) permite consolidar un pipeline analítico eficiente. La decisión de optimizar los cálculos matemáticos probó ser un camino técnico adecuado. Esto permitió procesar conjuntos masivos de datos y obtener indicadores críticos, como el Coeficiente de Edificabilidad (FAR), sin sobrecargar la infraestructura física local ni requerir extensiones de motor de sólidos más complejas.

Finalmente, la separación del sistema en canales independientes (un canal analítico liviano vectorizado en GeoJSON y un canal geométrico bajo demanda en 3D Tiles) garantiza que una plataforma de esta magnitud sea capaz de ofrecer una interfaz visual fluida y con alta capacidad de respuesta para el usuario, validando la arquitectura lógica propuesta.

9.2 Trabajo a Futuro

Si bien el prototipo demostró su solidez a nivel técnico y conceptual, dado el tiempo estipulado al calendario del curso impidieron profundizar en ciertas líneas de desarrollo. En caso de dar continuidad al proyecto, o si se quisiera realizar una transferencia tecnológica para su uso en una Intendencia o gobierno departamental, se identifican las siguientes líneas de trabajo a futuro:

- **Integración de un Motor de Sólidos Real (SFCGAL):** Con el fin de evaluar normativas urbanísticas aún más complejas, como las afectaciones de fachadas por planos límites inclinados, servidumbres de vistas o la proyección exacta de sombras en el espacio público, se plantea como evolución natural la incorporación de la extensión PostGIS SFCGAL. Esto habilitará operaciones booleanas 3D reales (ST_3DIntersects, ST_3DVolume) sobre los volúmenes del estándar CityGML.
- **Implementación de un Pipeline de Actualización Incremental:** El flujo de ingesta actual está diseñado como un proceso por lotes (*batch*). Para un entorno de producción municipal, se requeriría desarrollar servicios web automatizados que actualicen la base de datos de manera incremental a medida que las oficinas de catastro aprueban nuevos planos de mensura o finales de obra, evitando tener que re-procesar los datasets completos.
- **Edición Dinámica de Normativas desde la Interfaz:** Un módulo de alta utilidad para los planificadores urbanos de

una intendencia sería la capacidad de realizar simulaciones en tiempo real. Esto permitiría que un usuario con permisos modifique visualmente el FAR permitido o la altura máxima de una manzana directamente sobre el visor web, recalculando al instante qué edificios quedarían en infracción bajo ese nuevo escenario hipotético.

- **Optimización del Proceso de Normalización Altométrica:** Automatizar e integrar herramientas de procesamiento de nubes de puntos (como LiDAR) o modelos digitales de terreno (MDT) institucionales de forma directa en el contenedor de datos. Esto optimizaría la corrección sistemática de las cotas verticales (z), eliminando por completo el problema operativo de las estructuras flotantes durante la fase de carga.

Referencias

- [1] 3D City Database Project Team. 2024. 3D CityDB v5.1.3 - Open Source 3D City Database. <https://www.3dcitydb.org/>.
- [2] Cesium GS, Inc. 2026. CesiumJS - Open-source JavaScript library for world-class 3D globes and maps. <https://cesium.com/platform/cesiumjs/>.
- [3] Chair of Geoinformatics, Technical University of Munich. 2021. 3D City Model of New York City in CityGML. <https://www.asg.ed.tum.de/gis/projekte/new-york-city-3d/>.
- [4] Docker, Inc. 2026. Docker Engine Documentation. <https://docs.docker.com/engine/>. Consultado en Junio de 2026.
- [5] IOGP Geomatics Committee. 2026. EPSG Dataset: EPSG::32118 - NAD83 / New York Long Island. https://epsg.org/crs_32118/NAD83-New-York-Long-Island.html. Consultado en Junio de 2026.
- [6] NYC Department of City Planning. 2026. Primary Land Use Tax Lot Output (MapPLUTO), Version 26v1. <https://www.nyc.gov/site/planning/data-maps/open-data/dwn-pluto-mappluto.page>.
- [7] Open Geospatial Consortium. 2012. *OGC City Geography Markup Language (CityGML) Encodings Standard Version 2.0.0*. Technical Report. Open Geospatial Consortium. <https://www.ogc.org/standards/citygml>
- [8] Open Geospatial Consortium. 2014. *OGC Web Feature Service 2.0 Interface Standard*. Technical Report. Open Geospatial Consortium. Consultado en Junio de 2026.
- [9] Open Geospatial Consortium. 2026. OGC Standards and Specifications. <https://www.ogc.org/standards/>. Consultado en Junio de 2026.
- [10] Open Source Geospatial Foundation. 2026. GeoServer v2.25 - Open source server for sharing geospatial data. <https://geoserver.org/>.
- [11] PostGIS Project Steering Committee. 2026. PostGIS 3.6 Spatial Database Extender for PostgreSQL. <https://postgis.net>. Consultado en Junio de 2026.
- [12] PostgreSQL Global Development Group. 2026. PostgreSQL 18.3 Documentation. <https://www.postgresql.org/docs/18/>. Consultado en Junio de 2026.